

LAB EXPERIMENT ASSIGNMENT-1

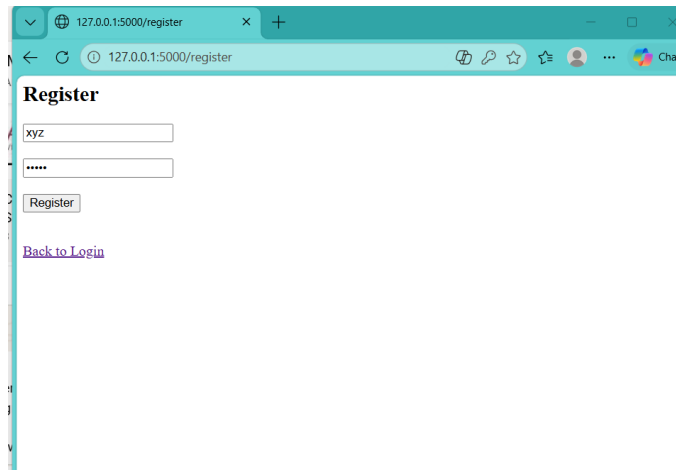
SECURE SYSTEM ENGINEERING

NAME: NIDHISHA S DHARAN

REG NO: CB.SC.P2CYS25019

Github repo: <https://github.com/Nidhisha7/Secure-Login-System->

Secure Login with bcrypt.



Register

xyz

.....

Register

[Back to Login](#)

Registration page

Login

Username

Password

Login

[Create Account](#)

Login page

Screenshot of db

	id	username	password_hash
1	5	test	\$2b\$12\$IVUJkub0ZgSKWHLUaZLG2uuU8NeX/qh2C1l0lc...
2	6	admin	\$2b\$12\$tkezuZZtPdlwPle/pjYecOY1DNsmu09M4qDzN9...

Credential stuffing.

```
import requests
```

```
# --- Configuration ---
```

```
TARGET_URL = "http://127.0.0.1:5000/login"
```

```
TARGET_USER = "admin"
```

```
WORDLIST = r"C:\Users\nidhi\Downloads\rockyou.txt"
```

```
def test_login_security():
```

```
    print(f"Testing login security for user: {TARGET_USER}")
```

```
    session = requests.Session()
```

```
    with open(WORDLIST, 'r', encoding='latin-1') as f:
```

```
        for line in f:
```

```
            password = line.strip()
```

```
            payload = {
```

```
                "username": TARGET_USER,
```

```
                "password": password
```

```
            }
```

```
            response = session.post(TARGET_URL, data=payload)
```

```
            if "Welcome" in response.text:
```

```
                print("\nLogin succeeded")
```

```
                print(f"Username: {TARGET_USER}")
```

```
                print(f"Password: {password}")
```

```
                return
```

```
            elif "ACCOUNT_LOCKED" in response.text:
```

```
                print("\nAccount locked")
```

```
                return
```

```
            else:
```

```
                print(f"Trying: {password} | Status: {response.status_code}", end="\r")
```

```
if __name__ == "__main__":
```

```
    test_login_security()
```

```
PS C:\Users\nidhi\secure-login-lab> python attack.py
Testing login security for user: admin
Trying: 123456789 | Status: 200
Login succeeded
Username: admin
Password: password
```

Same attack done after adding rate limiting and account lockout.

```
PS C:\Users\nidhi\secure-login-lab> python attack.py
Testing login security for user: admin

Account locked
```

TOTP-based 2FA

CODE:

```
from flask import Flask, request, render_template_string, redirect, session
import sqlite3
import bcrypt
import time
import pyotp
import qrcode
import os
```

```
app = Flask(__name__)
app.secret_key = "secret123"
```

```
DB_NAME = "users.db"
LOCK_LIMIT = 3
LOCK_TIME = 60
```

```
last_used_otp = {}
```

```
def init_db():
    con = sqlite3.connect(DB_NAME)
    cur = con.cursor()

    cur.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

        username TEXT UNIQUE NOT NULL,
        password_hash TEXT NOT NULL,
        totp_secret TEXT,
        failed_attempts INTEGER DEFAULT 0,
        locked_until INTEGER DEFAULT 0
    )
    """

```

```

con.commit()
con.close()

```

```

init_db()

```

```

@app.route("/")
@app.route("/login", methods=["GET"])
def login_page():
    return """
    <h2>Login </h2>
    <form method="POST" action="/login">
        <input name="username" placeholder="Username" required><br><br>
        <input name="password" type="password" placeholder="Password"
required><br><br>
        <button type="submit">Next</button>
    </form>
    <br>
    <a href="/register">Create Account</a>
    """

```

```

#Registration page

```

```

@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]

        hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()

    try:

```

```

con = sqlite3.connect(DB_NAME, timeout=5)
cur = con.cursor()

cur.execute(
    "INSERT INTO users (username, password_hash) VALUES (?, ?)",
    (username, hashed)
)
con.commit()

except sqlite3.IntegrityError:
    return "User already exists"

except Exception as e:
    return f"Error: {e}"

finally:
    con.close()

session["user"] = username
return redirect("/setup_2fa")

return """
<h2>Register</h2>
<form method="POST">
    <input name="username" required><br><br>
    <input name="password" type="password" required><br><br>
    <button type="submit">Register</button>
</form>
"""

#2FA setup page
@app.route("/setup_2fa", methods=["GET", "POST"])
def setup_2fa():
    if "user" not in session:
        return redirect("/login")

    username = session["user"]

```

```

con = sqlite3.connect(DB_NAME)
cur = con.cursor()

cur.execute(
    "SELECT totp_secret FROM users WHERE username=?",
    (username,)
)
row = cur.fetchone()

if row and row[0]:
    secret = row[0]
else:
    secret = pyotp.random_base32()
    cur.execute(
        "UPDATE users SET totp_secret=? WHERE username=?",
        (secret, username)
    )
    con.commit()

con.close()

totp = pyotp.TOTP(secret)

if not os.path.exists("static"):
    os.makedirs("static")

uri = totp.provisioning_uri(name=username, issuer_name="SecureApp")
img = qrcode.make(uri)
img.save("static/qrcode.png")

if request.method == "POST":
    otp = request.form["otp"]

    if totp.verify(otp):
        return redirect("/login")
    else:
        return "Invalid OTP during setup"

```

```

return f"""
<h2>Setup 2FA</h2>
<p>Scan using Google Authenticator</p>
<br><br>

<form method="POST">
    Enter OTP: <input name="otp">
    <button type="submit">Verify & Activate</button>
</form>
"""

```

#Login page

```
@app.route("/login", methods=["POST"])
```

```
def login():
```

```
    username = request.form["username"]
```

```
    password = request.form["password"]
```

```
    con = sqlite3.connect(DB_NAME, timeout=5)
```

```
    cur = con.cursor()
```

```
    cur.execute(
```

```
        "SELECT password_hash, failed_attempts, locked_until FROM users WHERE  
username=?",
```

```
        (username,)
    )
```

```
    user = cur.fetchone()
```

```
    if not user:
```

```
        time.sleep(1)
```

```
        return "Invalid username or password"
```

```
    password_hash, attempts, locked_until = user
```

```
    now = int(time.time())
```

```
    if locked_until > now:
```

```
        return "ACCOUNT_LOCKED"
```

```

if bcrypt.checkpw(password.encode(), password_hash.encode()):
    cur.execute(
        "UPDATE users SET failed_attempts=0, locked_until=0 WHERE username=?",
        (username,)
    )
    con.commit()

    session["temp_user"] = username
    return redirect("/verify_2fa")

```

```

attempts += 1

```

```

if attempts >= LOCK_LIMIT:
    locked_until = now + LOCK_TIME
    cur.execute(
        "UPDATE users SET failed_attempts=?, locked_until=? WHERE username=?",
        (attempts, locked_until, username)
    )
    con.commit()
    return "ACCOUNT_LOCKED"

```

```

cur.execute(
    "UPDATE users SET failed_attempts=? WHERE username=?",
    (attempts, username)
)
con.commit()

```

```

time.sleep(1)
return "Invalid username or password"

```

#2FA verification page

```

@app.route("/verify_2fa", methods=["GET", "POST"])

```

```

def verify_2fa():

```

```

    if "temp_user" not in session:
        return redirect("/login")

```

```

    username = session["temp_user"]

```

```

con = sqlite3.connect(DB_NAME)
cur = con.cursor()

cur.execute(
    "SELECT totp_secret FROM users WHERE username=?",
    (username,)
)
row = cur.fetchone()
con.close()

if not row or not row[0]:
    return "2FA not setup"

secret = row[0]
totp = pyotp.TOTP(secret)

message = ""

if request.method == "POST":
    otp = request.form["otp"]

    if last_used_otp.get(username) == otp:
        message = "Replay attack detected"
    elif not totp.verify(otp):
        message = "Invalid OTP"
    else:
        last_used_otp[username] = otp
        session["user"] = username
        session.pop("temp_user", None)
        return redirect("/welcome")

return f"""
<h2>Two-Factor Authentication</h2>

<form method="POST">
    Enter 6-digit code: <input name="otp"><br><br>
    <button type="submit">Verify</button>

```

```
</form>
```

```
<script>
```

```
    var msg = "{message}";
```

```
    if(msg) {{
```

```
        alert(msg);
```

```
    }}
```

```
</script>
```

```
"""
```

```
#welcome page
```

```
@app.route("/welcome")
```

```
def welcome():
```

```
    if "user" not in session:
```

```
        return redirect("/login")
```

```
    return f"<h2>Welcome, {session['user']}!</h2>"
```

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```

The application contains

- Register Page (/register)
- Setup 2FA Page (/setup_2fa)
- Login Page (/login)
- OTP Verification Page (/verify_2fa)
- Welcome Page (/welcome)

127.0.0.1 127.0.0.1:5000/register

127.0.0.1:5000/register Summarize

Register

127.0.0.1 127.0.0.1:5000/setup_2fa

127.0.0.1:5000/setup_2fa Summarize

Setup 2FA

Scan using Google Authenticator



Secret: DT7Z5GWW5EFBKQHZDJ5M4JQCNZO7WVP

Enter OTP:

127.0.0.1 127.0.0.1:5000/login

127.0.0.1:5000/login Summarize

Login

demo

.....

Next

[Create Account](#)

127.0.0.1 127.0.0.1:5000/verify_2fa

127.0.0.1:5000/verify_2fa Summarize

Two-Factor Authentication

Enter 6-digit code: 282895

Verify

127.0.0.1 127.0.0.1:5000/welcome

127.0.0.1:5000/welcome Summarize

Welcome, demo!

For wrong otp

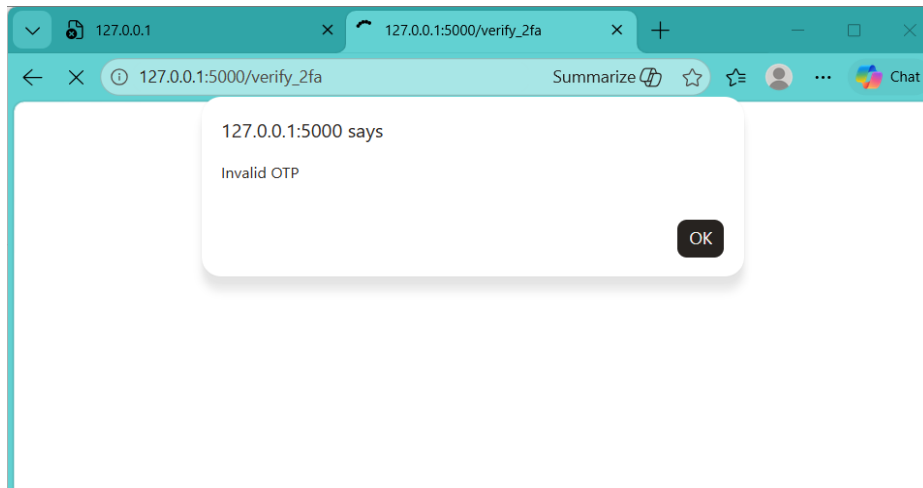
127.0.0.1 127.0.0.1:5000/verify_2fa

127.0.0.1:5000/verify_2fa Summarize

Two-Factor Authentication

Enter 6-digit code: 00000

Verify



```
PS C:\Users\nidhi\secure-login-lab> python attack.py
Testing login security for user: demo
Trying: 12345 | Status: 2000
Account locked
```

After implementing rate limiting and account lockout, the number of login attempts was restricted. The attack was blocked and no user accounts were compromised.